

UNIDAD 2

Gestión Lean-Agile de Productos de Software

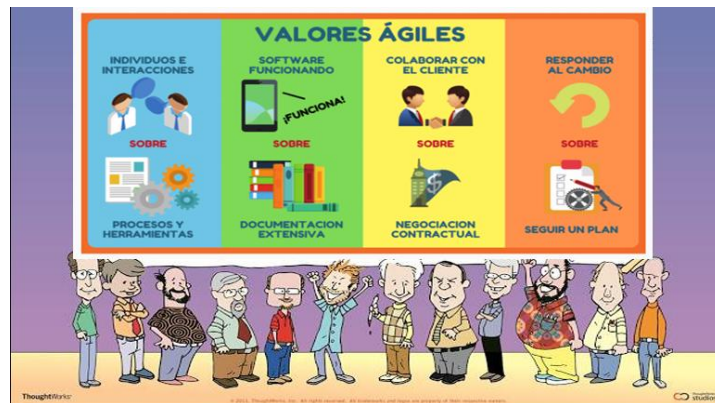
El Manifiesto Ágil

Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software. Este documento marcó un punto de inflexión al proponer un enfoque centrado en la entrega de valor, la adaptabilidad y las personas, en contraposición a los métodos tradicionales que se basaban en planes rígidos y procesos documentales pesados.

Ágil es una ideología con un conjunto definido de PRINCIPIOS que guían el desarrollo del producto.

Valores Ágiles

El Manifiesto Ágil expresa los siguientes valores. Es importante aclarar que Ágil no dice que los procesos, las herramientas, la documentación, los contratos o los planes sean inútiles o deban eliminarse. Lo que plantea es un **cambio de prioridades**.



1) Individuos e interacciones, por sobre procesos y herramientas

O en palabras más simples “Las herramientas no construyen el producto, la gente sí”.

El manifiesto plantea que el desarrollo de software es una actividad humano-intensiva. En tanto:

Se prefiere un equipo de **PERSONAS MOTIVADAS** con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos **desmotivados o poco funcional** con las mejores herramientas y procesos. Ágil lo que dice es que: Hay que dejar que quien tiene más experiencia en frontend lidere esas decisiones, sin forzar rotaciones artificiales solo por "estandarizar". También indica que **las herramientas se adaptan al equipo, no al revés**. Esto es clave y a veces se pierde de vista. Muchas organizaciones eligen una herramienta (por ejemplo, un framework de gestión o un software de tracking) y luego fuerzan al equipo a encajar en su forma de trabajar, incluso cuando eso genera fricción. El enfoque ágil invierte esa lógica: primero se entiende cómo el equipo trabaja mejor, y luego se buscan o configuran las herramientas que apoyen ese flujo natural, no que lo impongan.

2) Software funcionando por sobre documentación extensiva

La idea central: al priorizar el software funcionando (por sobre la documentación), el equipo entrega, al final de cada iteración, una versión del producto que:

- **Estable:** funciona correctamente, sin romper lo que ya existía.
- **Incremental:** suma funcionalidades nuevas sobre lo ya construido, en vez de entregar todo junto al final.
- **Mejorada:** cada iteración no solo agrega, sino que también pule o corrige lo anterior.

¿Para qué sirve esto? Porque al tener algo tangible y funcionando cada pocas semanas (no solo un documento), el equipo puede **mostrárselo al cliente o usuario final** y recibir su opinión real sobre lo que ve.

¿Por qué importa ese feedback? Porque le permite al equipo confirmar, iteración tras iteración, que está invirtiendo su tiempo en las funcionalidades que **realmente le importan al cliente**, y no en cosas que se asumieron como valiosas al principio del proyecto pero que en la práctica no cumplen sus expectativas.

Un documento de 40 páginas de requerimientos vs. un prototipo navegable: si el objetivo es que el cliente entienda cómo va a funcionar el checkout de un e-commerce, un prototipo interactivo (aunque sea con datos falsos) genera más retroalimentación real en 10 minutos de demo que semanas leyendo un documento de especificación funcional.

El enfoque ágil no elimina la documentación, sino que la hace **selectiva. Se documenta solo lo que agrega valor.** Si un documento no aporta algo útil y centrado en el cliente, no vale la pena escribirlo. Cabe aclarar si existe información necesaria de mantener ya que todo proyecto de software requiere cierto registro sobre el producto y el proceso (por ejemplo, decisiones de arquitectura, requerimientos clave, etc.). **Ágil no niega eso**, sino que la diferencia está en el "cuándo" en vez de documentar todo por adelantado "por las dudas", se documenta **cuando realmente se necesita**, en el momento en que ese documento va a ser útil. Entonces, de esto modo, la documentación se reduce a lo esencial, aunque esto no significa "cero documentaciones".

Otro aporte importante que nos enseña el enfoque ágil es que ningún documento, por bueno que sea, puede replicar la riqueza de una conversación cara a cara o de ver el software funcionando en vivo.

3) Colaboración con el cliente por sobre negociación contractual

En los enfoques tradicionales, es habitual que cliente y equipo de desarrollo firmen un **CONTRATO** al inicio del proyecto que fija **alcance, plazo y costo** (lo que se conoce como el "triángulo de hierro": scope, tiempo y presupuesto cerrados). El supuesto es que, si se negocia bien ese contrato al principio, ambas partes están protegidas. Claramente el problema es que el software se construye en un contexto de **incertidumbre y cambio constante**: el negocio evoluciona, el cliente entiende mejor su propio problema a medida que ve el producto avanzar, y el mercado cambia. Un contrato rígido no tiene forma de absorber esa realidad sin generar conflicto.

Cuando el cliente dice "me olvidé de esto" o "ahora necesito esto otro", después de haber firmado un contrato con alcance cerrado, ahí nace la **FRICCIÓN**: el equipo dice "eso no estaba en el contrato" y el cliente dice "pero lo necesito igual". Ambos tienen razón desde su posición, y esa tensión desgasta la relación. El contrato, que se firmó para dar seguridad, termina siendo un obstáculo.

¿Qué propone el enfoque Ágil?

Product Owner integrado al equipo: en Scrum, el cliente (o su representante) no espera a que termine el proyecto para opinar; participa activamente en la priorización del backlog, en el refinamiento de historias, y en cada **Sprint Review**. No es un actor externo que aprueba o rechaza al final, es parte del equipo.

Contratos ágiles o de "alcance variable": en vez de fijar 50 funcionalidades específicas por un monto cerrado, algunos equipos negocian contratos que fijan tiempo y presupuesto, pero dejan el alcance flexible ("trabajamos juntos 6 meses, priorizando lo que más valor te aporte en cada momento"). Esto permite que, si aparece una nueva prioridad, se pueda incorporar sin necesidad de renegociar todo el contrato.

Feedback en cada sprint en vez de una única aprobación final: si el cliente prueba el software cada 2 semanas y da su opinión, los cambios de rumbo se hacen a tiempo y de forma económica. Si el cliente solo ve el producto al final (como marca el contrato), cualquier cambio implica retrabajo masivo y renegociación.

Cliente presente en las pruebas: no se trata de mandarle un mail con capturas de pantalla, sino de que el cliente use el software real, lo pruebe con datos reales, y dé **feedback directo**. Eso genera compromiso genuino con el resultado, no solo una firma de aceptación.

4) Responder a Cambios por sobre Seguir un Plan

La gestión tradicional de proyectos parte de un supuesto: si al inicio se define bien el plan (requerimientos, cronograma, alcance), el trabajo consiste en **ejecutar ese plan y controlar las desviaciones**. Cualquier cambio se ve como un problema, algo que hay que evitar o, si ocurre, gestionar como una excepción costosa (change request, aprobación, renegociación).

El problema de fondo surge debido a que **es imposible que un cliente conozca con certeza absoluta todas sus necesidades al comienzo de un proyecto**.

- El propio cliente entiende mejor lo que necesita **a medida que ve el software funcionando**, no antes.
- El mercado, la competencia o la regulación pueden cambiar durante el desarrollo.

En ágil, el plan **no desaparece**, pero deja de ser un documento fijo escrito al principio: se convierte en algo vivo, que se ajusta constantemente en base a lo que se va aprendiendo y ese conocimiento no aparece solo al principio del proyecto, sino que se va generando **iteración tras iteración**.

Este valor **no significa "no planificar"**, así como el valor anterior no significaba "no documentar". Ágil sigue planificando, pero lo hace de forma **incremental y adaptativa**: se planifica con el nivel de detalle adecuado para el corto plazo, y con mayor flexibilidad para el largo plazo.

Principios del Manifiesto Ágil



1. Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor

Explicación fundamentada:

Esto se soporta mediante el uso de *User Stories* (Historias de Usuario). Una Historia de Usuario obliga al equipo a programar "porciones verticales" del sistema que el usuario pueda utilizar (interfaz, lógica y base de datos de una funcionalidad específica), en lugar de programar tareas técnicas horizontales (como "crear todas las tablas de la base de datos", lo cual no aporta valor directo e inmediato al cliente). Al entregar las Historias de Usuario de mayor prioridad primero, el cliente puede empezar a utilizar el software y obtener beneficios económicos mucho antes de que el sistema completo esté terminado.

Ejemplo Práctico:

- En el desarrollo de una tienda virtual, en lugar de que los programadores dediquen un mes entero a configurar la arquitectura de TODA la base de datos (una tarea técnica sin valor visible para el usuario), el equipo toma la Historia de Usuario de mayor prioridad: "Como comprador, quiero pagar con tarjeta de crédito para recibir mi pedido en casa". En una iteración, el equipo programa la interfaz de pago, la lógica de validación y las tablas de base de datos **SOLO** para esa funcionalidad. Al finalizar, el cliente ya tiene una tienda capaz de procesar ventas reales y generar ingresos, satisfaciendo su necesidad principal de forma temprana.

2. Aceptar que los requisitos cambien, incluso en etapas tardías del desarrollo

La ingeniería de software tradicional asume que el costo de realizar un cambio crece exponencialmente a medida que avanza el proyecto, por lo que exige "congelar" los requerimientos al inicio. La bibliografía Lean y Ágil demuestra que esto genera un gran desperdicio, ya que obliga a tomar decisiones cuando se tiene la menor cantidad de información disponible.

Lo que lo distingue de la gestión tradicional es el matiz "incluso en etapas tardías". No alcanza con estar abierto al cambio al principio del proyecto (eso cualquier metodología lo acepta); la diferencia real está en aceptar el cambio cuando ya se avanzó bastante, sin tratarlo como una crisis o una falla del proceso.

Ejemplo concreto: si en la iteración 8 de 12 el cliente pide cambiar la lógica de cálculo de precios porque cambió una regulación impositiva, un equipo tradicional (con alcance cerrado por contrato) lo trataría como un cambio costoso y conflictivo. Un equipo ágil lo incorpora al backlog, lo prioriza según su urgencia real, y lo desarrolla en las próximas iteraciones, entendiendo que ese cambio es tan legítimo como cualquier requerimiento original.

Esto es posible justamente porque, como vimos antes, no hay un plan rígido de 6 meses firmado en piedra: hay un backlog vivo que se reordena constantemente.

3. Entregar software funcional frecuentemente (entre dos semanas y dos meses, con preferencia al periodo más corto)

En el desarrollo de software, el trabajo parcialmente terminado (código escrito, pero no probado, o probado, pero no integrado) es un desperdicio que oculta errores graves. Agile soluciona esto dividiendo el trabajo en ciclos de tiempo fijos y cortos llamados iteraciones o *Sprints* (generalmente de 1 a 4 semanas).

Al final de cada iteración el equipo no puede entregar código "al 90%". El software debe cumplir rigurosamente con la **Definición de Hecho** (*Definition of Done - DoD*), lo que significa que el código debe estar escrito, refactorizado, probado (pruebas unitarias y de integración pasadas) y sin errores conocidos. El resultado debe ser un **incremento potencialmente entregable** (*Potentially Shippable Product Increment*); es decir, una pieza de software con una calidad tan alta que el cliente podría decidir ponerlo en los servidores de producción ese mismo día si así lo deseara.

FILOSOFÍA ÁGIL

¿Por qué existe el enfoque ágil? y qué lo diferencia estructuralmente de otros modelos iterativos. Vamos por partes.

El objetivo primordial: entregar software frecuente en un entorno cambiante

¿por qué existe ágil?

El objetivo primordial: entregar software frecuente en un entorno cambiante

Ágil surge como respuesta a proyectos donde los requerimientos no se pueden fijar con certeza al principio, por dos razones concretas:

1. El cliente no puede predecir el impacto real del sistema hasta que lo usa en su operación diaria.
2. El entorno de negocio cambia durante el desarrollo (mercado, competencia, regulaciones), y eso puede volver obsoleto algo que tenía sentido al comienzo del proyecto.

¿cómo responde ágil a ese problema?

El ciclo iterativo-incremental

- **Iterativo:** el trabajo se hace en ciclos repetidos, no de una sola vez.
- **Incremental:** cada ciclo suma una porción nueva y funcional al sistema ya construido.

El concepto clave que sostiene ese ciclo: el Empirismo

El conocimiento se construye a partir de la **experiencia y observación real**, no de la planificación teórica anticipada.

El equipo no asume que sabe de antemano cómo va a resultar el sistema completo; lo **descubre iteración a iteración**, observando resultados reales.

Por qué esto distingue a ágil de otros modelos iterativos (PUD/RUP)

Este es el punto más sutil: **PUD/RUP también son iterativos-incrementales**, pero son **procesos definidos**, no empíricos.

	Proceso Definido (PUD/RUP)	Proceso Empírico (Ágil)
Supuesto de base	Se puede predecir el resultado siguiendo el proceso correctamente	Hay demasiada incertidumbre para predecir; hay que descubrir sobre la marcha
Estructura	Iterativo, pero con fases y artefactos bastante definidos de antemano	Iterativo, pero flexible en su ejecución

Es decir: ser iterativo **no alcanza** para ser ágil. Lo que define a ágil es aceptar que **ni siquiera el proceso mismo puede especificarse del todo de antemano**, porque la incertidumbre es demasiado alta.

Dos consecuencias prácticas de este mecanismo

a) El cliente inmerso en el proceso

Los incrementos (típicamente cada 2-3 semanas) se ponen a disposición del cliente para obtener feedback real. **El cliente no espera pasivamente al final; participa activamente inspeccionando cada incremento y ayudando a decidir el rumbo del siguiente.** *(Conecta con el valor 3 del manifiesto: colaboración con el cliente.)*

b) Comunicación informal en lugar de documentación formal

No es solo "documentar menos": es **reemplazar** documentación por comunicación frecuente e informal (daily standups, charlas, mensajes rápidos) **en vez de reuniones formales con actas y documentos firmados.** *(Conecta con el valor 2 del manifiesto: software funcionando sobre documentación extensiva.)*

¿qué tan estructurado debe ser ágil?

La definición de Martin Fowler

"Un compromiso entre nada de proceso y demasiado proceso."

Extremo	Qué pasa	Resultado
Demasiado proceso (tradicional)	El proceso se vuelve un fin en sí mismo: importa cumplir el checklist, llenar el documento, seguir la ceremonia	El proceso se antepone al resultado
Nada de proceso	Programar sin disciplina, sin planificación ni revisión	Caos, falta de previsibilidad, calidad inconsistente
Ágil (punto medio)	Suficiente estructura para ser predecible y sostenible (sprints, ceremonias, roles), sin caer en rigidez burocrática	El proceso sirve al equipo y al resultado, no al revés

La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales, la transparencia, la inspección y la adaptación.

Los 3 pilares del empirismo

Este cierre es clave porque son los pilares que sostienen técnicamente todo lo anterior (son, de hecho, los mismos pilares que después formaliza Scrum):

1. **Transparencia:** todos los aspectos relevantes del proceso deben ser visibles para quienes son responsables del resultado. Ejemplo: un backlog visible para todo el equipo y el cliente, no información oculta o repartida en silos.
2. **Inspección:** se debe revisar frecuentemente el progreso hacia el objetivo, para detectar variaciones o problemas. Ejemplo: la Sprint Review, donde se inspecciona el incremento construido.
3. **Adaptación:** si la inspección revela que algo se está desviando de lo aceptable o de lo que el cliente necesita, hay que ajustar el proceso o el producto lo antes posible. Ejemplo: la Retrospectiva, donde el equipo ajusta su forma de trabajar en base a lo aprendido.

Sin estos tres pilares funcionando juntos, el ciclo iterativo-incremental pierde sentido: se podría iterar sin aprender nada, si no hay transparencia real o si nadie inspecciona y adapta en base a lo que se observa.

¿Cuándo es aplicable Ágil?

Este texto responde una pregunta distinta a las anteriores: no "qué es ágil" sino "en qué contextos tiene sentido usarlo". Es un tema clave porque ágil no es una solución universal para cualquier tipo de proyecto.

Ágil no es un framework, es una ideología

Este primer punto es fundamental y muchas veces se confunde en la práctica:

- **Ágil** = un conjunto de valores y principios (el Manifiesto). Define un *para qué* y un *cómo pensar*, no un *cómo hacer* paso a paso.
- **Los frameworks ágiles** (Scrum, Kanban, XP, SAFe, etc.) = implementaciones concretas de esa ideología. Son los que sí definen roles, ceremonias, artefactos específicos.

Ejemplo: decir "somos ágiles" no dice mucho por sí solo. Decir "usamos Scrum con sprints de 2 semanas" ya es una implementación concreta de esa ideología ágil.

Un matiz importante: gestión vs. ingeniería

La mayoría de las prácticas ágiles son de gestión de proyectos, no de ingeniería de software. Scrum, por ejemplo, te dice cómo organizar el trabajo del equipo (sprints, backlog, roles), pero no te dice cómo programar bien.

El modelo de certeza/entendimiento: ¿dónde funciona mejor ágil?

Esta es la parte central del texto. Se basa en un modelo (conocido como la Matriz de Stacey o modelos similares) que clasifica los problemas

Zona	Características	Enfoque recomendado
Simple	Alta certeza y alto entendimiento. Se sabe qué hay que hacer y cómo hacerlo.	Procesos predictivos/tradicionales funcionan bien
Complicado	Requiere expertise, pero es resoluble con análisis	Procesos más estructurados, con expertos
Complejo	Entendimiento y certeza parciales. Se sabe algo, pero hay que ir descubriendo el resto sobre la marcha	Acá es donde ágil da mejores resultados
Caótico / "Anarquía"	Gran incertidumbre, casi no hay entendimiento previo	Investigación, experimentación exploratoria (no ágil "puro" de entrega)

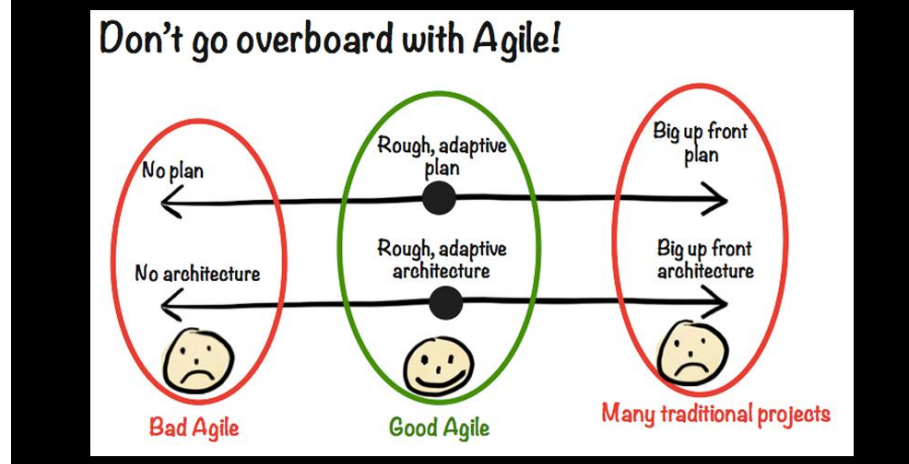
Por qué ágil brilla específicamente en "lo complejo"

Un **producto nuevo** es el ejemplo típico de este cuadrante: hay una idea de negocio, hay cierto entendimiento del problema, pero no hay certeza total sobre qué funcionalidades exactas se van a necesitar ni cómo van a reaccionar los usuarios. Es demasiada incertidumbre para planificar todo de antemano (por eso no aplica bien un enfoque predictivo tradicional), pero hay suficiente entendimiento como para poder iterar y aprender con cada incremento (por eso no es pura "anarquía" tampoco).

"Las metodologías ágiles son aplicables en el contexto de lo complejo (dan buenos resultados), en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial."

Ejemplo: desarrollar una nueva app de fintech para un mercado que recién se está formando. No hay certeza absoluta sobre qué funcionalidades van a adoptar los usuarios, pero tampoco se está totalmente a ciegas: hay hipótesis de negocio razonables que se pueden ir validando iteración a iteración. Ese es el terreno ideal para ágil.

Ser Ágil no es ser indisciplinado.



"Ser ágil no es ser indisciplinado"

Mucha gente interpreta "ágil" como sinónimo de "trabajar sin estructura, improvisando sobre la marcha, sin compromiso ni disciplina". Es exactamente lo contrario de lo que plantea la filosofía ágil.

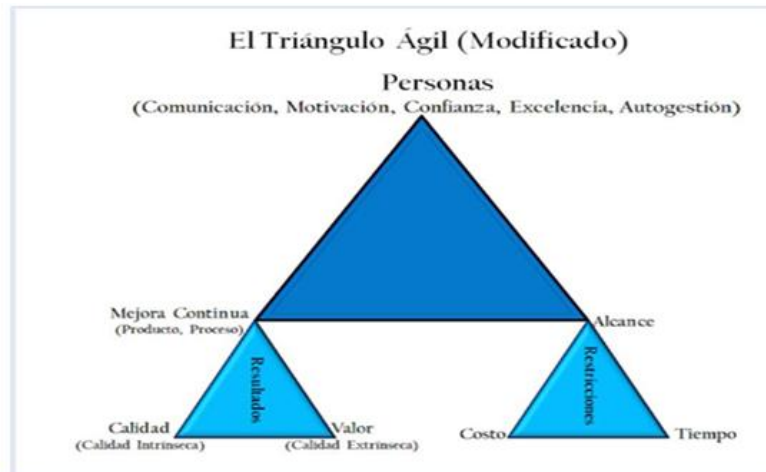
Como vimos antes con la cita de Fowler ("un compromiso entre nada de proceso y demasiado proceso"), ágil **requiere** disciplina: cumplir con la Definition of Done, respetar los tiempos de las ceremonias, mantener la transparencia del backlog, testear antes de dar algo por terminado. La flexibilidad está en el *qué* se construye y en *qué orden* (adaptación continua), no en la *falta de rigor* con la que se trabaja.

¿Por qué ir a Ágil?

- **Acelera entregas de valor:** al trabajar en iteraciones cortas, el cliente recibe funcionalidades usables cada pocas semanas, no al final del proyecto.
- **Adaptación a cambios:** el backlog se Re prioriza constantemente, así que un cambio de requerimiento se absorbe en la próxima iteración, sin romper todo el plan.
- **Alineación entre negocio y TI:** al tener al cliente/PO priorizando según valor real de negocio, el equipo técnico no trabaja en cosas desconectadas de los objetivos de la empresa.
- **Reduce riesgos:** entregar seguido con feedback temprano permite detectar errores (técnicos o de negocio) cuando todavía son baratos de corregir, no al final del proyecto.
- **Mejor visibilidad:** gracias a la transparencia (backlog visible, demos frecuentes), todos saben el estado real del proyecto, sin depender de reportes desactualizados.

- **Mejora relaciones de equipo:** la comunicación fluida y la colaboración constante (en vez de silos) fortalecen la confianza entre el equipo y con el cliente.
- **Reduce costos:** al detectar errores o cambios de rumbo temprano, se evita el retrabajo masivo de corregir algo cuando ya está todo construido sobre una base equivocada.

El Triángulo Ágil



El triángulo ágil apunta a un producto de software que pone a las **personas** (que tienen ciertas características como confianza, motivación, etc.) en el lugar más importante ya que las personas son las que hacen el software. Donde se prioriza que haya comunicación, motivación, confianza, excelencia y autogestión.

Debajo encontramos el pie, en el cual tenemos la **mejora continua** (tanto del **producto** (vía iteraciones) como del **proceso** (vía comunicación del equipo)). Es el triángulo de resultados, teniendo como pie a la calidad la cual es calidad intrínseca, las características técnicas que el equipo le da al producto (buen código, bien construido), y la calidad extrínseca, el impacto real que el producto tiene para el cliente.

Del otro lado de la base podemos encontrar el **alcance**, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.

Finalmente se tienen en cuenta las variables que se usan para gestionar el trabajo (alcance- costo-tiempo).

Debemos hacer foco en entregarle al cliente un producto de calidad que se vaya mejorando y evolucionando en forma continua.

Claramente cambia la pregunta central del proyecto: en vez de "¿cumplimos alcance, costo y tiempo?" (triángulo de hierro tradicional), pregunta "**¿esto genera calidad y valor real para el cliente?**"

Alcance, costo y tiempo **siguen existiendo**, pero pasan a ser **restricciones**, no el objetivo.

Introducción al Desarrollo Ágil